

EURON 10주차 발표자료 요약

자연어 처리(NLP, Natural Language Processing)는 인간의 언어를 컴퓨터가 이해하고 처리할 수 있도록 하는 기술이다. 인간의 언어는 본질적으로 모호성, 가변성, 구조성이라는 세 가지 특징을 지니는데, 하나의 문장이 여러 의미로 해석될 수 있고 같은 의미도 다양한 표현으로 나타나며, 동시에 문법과 규칙이라는 구조를 따른다. 이러한 특성 때문에 컴퓨터는 인간의 언어를 그대로 이해할 수 없으며, 반드시 텍스트를 숫자 형태로 변환하는 과정이 필요하다. 이때 텍스트를 처리 가능한 형태로 바꾸는 첫 단계가 바로 토큰화(Tokenization)이다.

토큰화란 텍스트를 의미 있는 최소 단위인 토큰으로 나누는 과정이다. 자연어 처리에서는 분석 대상이 되는 텍스트 집합을 말뭉치(Corpus)라고 하며, 토큰화 이후 나뉜 개별 단위를 토큰(Token)이라고 한다. 이러한 토큰화를 수행하는 규칙이나 모델을 토큰나이저(Tokenizer)라고 한다. 예를 들어 “나는 NLP를 공부한다”라는 문장은 토큰화 과정을 통해 ['나는', 'NLP를', '공부한다']와 같이 분리될 수 있다. 중요한 점은 토큰나이저의 방식에 따라 결과가 달라질 수 있다는 것이다.

토큰나이저를 구축하는 방법에는 여러 가지가 있다. 가장 간단한 방법은 공백을 기준으로 나누는 방식이지만, 이는 한국어나 중국어처럼 띄어쓰기 규칙이 복잡한 언어에는 적합하지 않다. 사전 기반 방식은 미리 정의된 단어 목록을 활용하는데, 사전에 없는 단어가 등장할 경우 처리하지 못하는 문제가 있다. 정규표현식 기반 방식은 특정 패턴을 이용하여 토큰을 분리하며, URL이나 특수문자 처리에 유용하다. 최근에는 머신러닝 기반 방법이 많이 사용되며, 데이터로부터 토큰화 규칙을 학습하는 방식으로 BPE나 WordPiece 같은 알고리즘이 대표적이다. 이 과정에서 중요한 문제 중 하나가 OOV(Out-Of-Vocabulary) 문제이다. 이는 학습 시 등장하지 않았던 단어가 입력으로 들어오는 경우를 의미한다. 예를 들어 ‘movie’, ‘good’, ‘bad’만 학습된 모델에 ‘masterpiece’라는 단어가 들어오면 이를 처리하지 못한다. 이러한 문제는 데이터 희소성(sparse data)과 차원의 저주를 유발한다. 따라서 이를 해결하기 위해 단어를 더 작은 단위로 나누는 방식이 필요하게 된다.

토큰화 방식에는 여러 종류가 존재한다. 먼저 단어 토큰화는 텍스트를 단어 단위로 나누는 가장 직관적인 방법이다. 그러나 ‘cg’와 ‘cg.’처럼 동일한 의미라도 형태가 다르면 서로 다른 토큰으로 인식되어 OOV 문제가 증가할 수 있다. 글자(문자) 토큰화는 단어를 다시 문자 단위로 분해하는 방식으로, 어휘의 크기를 줄이고 OOV 문제를 완화할 수 있지만 의미 정보가 부족하다는 단점이 있다. 특히 한국어에서는 자모 단위로 분해하여 초성, 중성, 종성을 활용하는 방식도 사용된다.

형태소 토큰화는 의미를 가진 최소 단위인 형태소를 기준으로 분리하는 방식으로, 한국어 처리에서 매우 중요하다. 예를 들어 “그는 나에게 인사를 했다”라는 문장은 ‘그’, ‘는’, ‘나’, ‘에게’, ‘인사’, ‘를’, ‘했’, ‘다’와 같이 나뉜다. 이 방식은 문법 정보를 반영할 수 있기 때문에 문맥 이해가 가능하며, 검색, 요약, 감성 분석 등의 작업에 유리하다. 이러한 형태소 분석을 위해 KoNLPy, NLTK와 같은 다양한 라이브러리가 사용된다.

하지만 기존의 형태소 분석 방식은 신조어나 오타자, 새로운 표현을 처리하는 데 한계가 있

다. 시간이 지남에 따라 언어는 계속 변화하고 새로운 단어가 등장하기 때문에, 사전 기반 접근만으로는 이를 모두 처리하기 어렵다. 이러한 문제를 해결하기 위해 등장한 것이 하위 단어(Subword) 토큰화이다. 하위 단어 토큰화는 하나의 단어를 더 작은 단위의 조합으로 나누어 처리하는 방식으로, OOV 문제를 완화하고 어휘 사전의 크기를 줄이며 처리 속도를 향상시키는 장점을 가진다.

대표적인 하위 단어 토큰화 알고리즘으로는 BPE(Byte Pair Encoding)가 있다. BPE는 입력 데이터에서 가장 자주 등장하는 글자 쌍을 찾아 반복적으로 병합하는 방식으로 동작한다. 자주 등장하는 단어는 하나의 토큰으로 유지되고, 덜 등장하는 단어는 여러 개의 토큰으로 분해된다. 이와 유사하지만 확률 기반으로 병합을 수행하는 방식이 WordPiece이다. WordPiece는 병합할 때 점수를 계산하여 가장 적절한 조합을 선택하며, BERT와 같은 모델에서 사용된다. 또한 SentencePiece는 구글에서 개발한 오픈소스 라이브러리로, 별도의 전처리 없이도 하위 단어 토큰화를 수행할 수 있으며 다양한 설정을 통해 유연하게 활용할 수 있다.

토큰화 이후에는 텍스트를 숫자 형태로 변환하는 임베딩 과정이 필요하다. 워드 임베딩은 단어를 벡터로 표현하는 방법으로, 의미가 비슷한 단어는 벡터 공간에서 가까운 위치에 놓이도록 한다. 초기에는 원-핫 인코딩이나 빈도 기반 벡터가 사용되었지만, 이는 희소성이 크고 의미 정보를 충분히 담지 못하는 한계가 있었다. 이후 Word2Vec, fastText와 같은 방법이 등장했지만, 이 역시 단어를 항상 동일한 벡터로 표현하기 때문에 문맥에 따라 의미가 달라지는 경우를 반영하기 어렵다. 이러한 문제를 해결하기 위해 최근에는 문맥을 반영하는 동적 임베딩이 사용된다.

언어 모델은 이러한 임베딩을 기반으로 다음 단어를 예측하는 모델이다. 자기회귀 언어 모델은 이전에 등장한 단어들을 바탕으로 다음 단어를 순차적으로 예측하며 문장을 생성한다. 반면 통계적 언어 모델은 단어의 등장 확률과 패턴을 이용하여 다음 단어를 예측하며, 대표적으로 마르코프 체인을 기반으로 한다. 하지만 이러한 모델은 긴 문맥을 이해하는 데 한계가 있다.

이러한 한계를 보완하기 위한 방법 중 하나가 N-gram 모델이다. N-gram은 연속된 N개의 단어를 묶어 다음 단어를 예측하는 방식으로, unigram, bigram, trigram 등이 있다. 이 방식은 비교적 간단하고 적은 데이터로도 패턴을 파악할 수 있지만, 이전 N-1개의 단어만 고려하기 때문에 문맥을 충분히 반영하지 못한다.

마지막으로 TF-IDF는 문서에서 중요한 단어를 추출하기 위한 방법이다. 이는 단어가 특정 문서에서 얼마나 자주 등장하는지(TF)와 여러 문서에서 얼마나 희귀한지(IDF)를 함께 고려하여 중요도를 계산한다. 즉, 특정 문서에서 자주 등장하지만 전체 문서에서는 드문 단어일수록 높은 중요도를 갖는다. 하지만 이 방식 역시 단어의 의미나 문맥을 반영하지 못한다는 한계가 있다.

종합적으로 보면 자연어 처리는 텍스트를 토큰화하고 이를 벡터로 변환한 뒤, 언어 모델을 통해 의미를 이해하고 예측하는 일련의 과정으로 이루어진다. 특히 토큰화는 이러한 전체 과정

의 출발점으로서 매우 중요한 역할을 한다. 최근에는 하위 단어 토큰화와 문맥 기반 임베딩 기술의 발전을 통해 기존의 한계를 극복하고 있으며, 이는 GPT나 BERT와 같은 현대 자연어 처리 모델의 핵심 기반이 되고 있다.